

Trabalho de Visão Computacional

Classificação de Placas de Trânsito com GTSRB

1 Objetivo

Neste trabalho, vocês deverão desenvolver e avaliar modelos de redes neurais convolucionais para classificação de imagens do dataset **GTSRB** — **German Traffic Sign Recognition Benchmark**.

O objetivo principal não é apenas obter alta acurácia global, mas investigar o comportamento do modelo **por classe**. O grupo deverá identificar quais classes são mais difíceis, analisar possíveis causas dos erros e propor experimentos para melhorar o desempenho dessas classes.

Ao final do trabalho, o grupo deverá ser capaz de responder perguntas como:

- Quais classes foram mais fáceis e mais difíceis?
- O modelo erra mais em classes com poucos exemplos?
- Quais classes são confundidas entre si?
- Batch normalization melhora o treinamento?
- SGD, SGD com momentum e Adam produzem resultados diferentes?
- Data augmentation melhora a acurácia global ou apenas algumas classes?
- Estratégias para lidar com desbalanceamento melhoram as classes com pior desempenho?
- O melhor modelo em acurácia global também é o melhor em macro accuracy?

2 Dataset

O dataset utilizado será o **GTSRB**, disponível no `torchvision.datasets.GTSRB`. O GTSRB contém imagens de placas de trânsito alemãs organizadas em **43 classes**. As imagens possuem variações de tamanho, iluminação, posição, nitidez e perspectiva. O grupo deverá redimensionar as imagens para um tamanho fixo. A escolha do tamanho da imagem deve ser explicada no relatório.

O carregamento do dataset deve seguir a separação oficial:

- `split="train"` para treinamento e validação;
- `split="test"` para avaliação final.

O conjunto de treinamento deverá ser dividido em treino e validação. O conjunto de teste deve ser usado apenas para avaliação final.

Todos os experimentos devem usar a mesma divisão treino/validação, para que os resultados sejam comparáveis.

3 Tarefa

A tarefa consiste em treinar modelos capazes de classificar corretamente a placa de trânsito presente em cada imagem.

Cada grupo deverá implementar:

1. Um modelo **baseline**;
2. Experimentos de melhoria;
3. Uma análise detalhada dos resultados por classe;
4. Um arquivo `.csv` contendo as predições do melhor modelo no conjunto de teste.

4 Modelo baseline obrigatório

O grupo deve implementar uma CNN simples, treinada do zero, contendo pelo menos:

- camadas convolucionais;
- função de ativação ReLU;
- pooling;
- uma ou mais camadas fully connected;
- função de perda `CrossEntropyLoss`;
- cálculo de acurácia no conjunto de validação e no conjunto de teste.

O baseline deve servir como referência para todos os outros experimentos.

Exemplo de arquitetura baseline aceitável:

```
Conv2d -> ReLU -> MaxPool
Conv2d -> ReLU -> MaxPool
Flatten
Linear -> ReLU
Linear -> saida com 43 classes
```

O grupo pode propor uma arquitetura baseline diferente, desde que ela seja simples, treinada do zero e claramente descrita no relatório.

5 Experimentos obrigatórios

Cada grupo deverá realizar, no mínimo, os seguintes experimentos.

5.1 Experimento 1 — Otimizadores

Comparar pelo menos três estratégias de otimização:

- SGD;
- SGD com momentum;
- Adam.

O grupo deve discutir:

- qual otimizador convergiu mais rápido;
- qual obteve maior acurácia global;
- qual teve melhor desempenho nas classes difíceis;
- se houve sinais de overfitting;
- se a escolha do otimizador afetou a estabilidade do treinamento.

5.2 Experimento 2 — Batch Normalization

Comparar uma arquitetura:

- sem batch normalization;
- com batch normalization.

O grupo deve analisar se o uso de batch normalization:

- acelerou o treinamento;
- melhorou a estabilidade;
- aumentou a acurácia global;
- melhorou a macro accuracy;
- melhorou a acurácia das classes com pior desempenho.

5.3 Experimento 3 — Data Augmentation

Aplicar pelo menos duas técnicas de data augmentation, como:

- pequenos deslocamentos;
- rotações leves;
- alterações de brilho;
- alterações de contraste;
- random crop;

- blur leve.

O grupo deve comparar o modelo treinado:

- sem data augmentation;
- com data augmentation.

A análise deve mostrar se a técnica ajudou todas as classes ou apenas algumas.

6 Experimentos opcionais

Além dos experimentos obrigatórios, o grupo pode realizar um ou mais experimentos extras.

6.1 Arquiteturas alternativas

Comparar a CNN baseline com:

- uma VGG-like pequena;
- uma ResNet pequena;
- uma arquitetura criada pelo próprio grupo.

Modelos pré-treinados não são permitidos. Todas as arquiteturas devem ser treinadas do zero.

6.2 Regularização

Testar estratégias como:

- dropout;
- weight decay;
- early stopping;

6.3 Desbalanceamento de classes

Testar estratégias como:

- `WeightedRandomSampler`;
- pesos na `CrossEntropyLoss`;
- oversampling de classes minoritárias;
- augmentation mais forte nas classes com pior desempenho.

A análise deve deixar claro se a técnica melhorou apenas a acurácia global ou se também melhorou o desempenho das classes difíceis.

6.4 Tamanho da imagem

Comparar o desempenho usando imagens redimensionadas para diferentes tamanhos. O grupo deve discutir o impacto do tamanho da imagem na acurácia, no custo computacional e no tempo de treinamento.

7 Métricas obrigatórias

O relatório deve apresentar as seguintes métricas:

Métrica	Descrição
Acurácia global	Percentual total de imagens classificadas corretamente.
Acurácia por classe	Acurácia individual para cada uma das 43 classes.
Macro accuracy	Média das acurácias por classe.
Pior classe	Classe com menor acurácia.
Melhor classe	Classe com maior acurácia.
Matriz de confusão	Mostra quais classes são confundidas entre si.
Curvas de treino	Loss e acurácia ao longo das épocas.

Apenas reportar acurácia global **não é suficiente**.

8 Análise por classe

Esta é uma das partes mais importantes do trabalho.

Após treinar o baseline, o grupo deverá identificar:

- as 5 classes com maior acurácia;
- as 5 classes com menor acurácia;
- as principais confusões observadas na matriz de confusão;
- possíveis razões para os erros;
- se as classes com pior desempenho possuem poucos exemplos no treinamento.

Em seguida, o grupo deverá propor pelo menos **duas hipóteses** para melhorar as classes com pior desempenho.

Exemplos de hipóteses:

```
Hipotese 1:
As classes com poucos exemplos estao tendo pior desempenho.
Vamos testar weighted loss para dar maior peso a essas classes.

Hipotese 2:
O modelo esta confundindo placas visualmente parecidas.
Vamos testar data augmentation e uma arquitetura mais profunda.

Hipotese 3:
```

```
O modelo esta sofrendo overfitting.
Vamos testar dropout e weight decay.
```

Cada hipótese deve ser acompanhada de um experimento e de uma análise dos resultados.

9 Salvamento dos resultados

Além das métricas apresentadas no relatório, cada grupo deverá salvar as predições do melhor modelo em um arquivo `.csv`. Esse arquivo será utilizado pelo professor para calcular métricas padronizadas e comparar os resultados entre os grupos.

Para garantir uma comparação justa, todos os grupos deverão utilizar o arquivo auxiliar fornecido pelo professor para carregar os dados e salvar as predições. Esse arquivo não deve ser modificado.

O arquivo auxiliar define:

- o carregamento do dataset GTSRB;
- a divisão fixa entre treino e validação, usando `seed=42`;
- a proporção de 80% para treino e 20% para validação;
- o uso do conjunto oficial de teste do GTSRB;
- o `DataLoader` de teste com `shuffle=False`;
- a função padronizada `save_predictions` para exportar o arquivo final.

Cada grupo deverá entregar obrigatoriamente um arquivo chamado:

```
NOME1_NOME2_RESULTS.csv
```

onde `NOME1` e `NOME2` devem ser substituídos pelo primeiro nome dos integrantes do grupo.

Por exemplo:

```
ANA_PEDRO_RESULTS.csv
```

Esse arquivo deve conter as predições do melhor modelo escolhido pelo grupo no conjunto de teste.

9.1 Formato obrigatório do arquivo

O arquivo de resultados deve ser gerado usando a função `save_predictions` fornecida no arquivo auxiliar.

O formato obrigatório do arquivo é:

O arquivo deve conter uma linha para cada imagem do conjunto de teste, seguindo exatamente a ordem do `DataLoader` de teste.

Como o conjunto oficial de teste do GTSRB possui 12.630 imagens, o arquivo final deve conter 12.630 linhas de predições, além do cabeçalho.

Exemplo de arquivo:

Coluna	Tipo	Descrição
image_index	inteiro	Índice da imagem no conjunto de teste.
predicted_class	inteiro	Classe prevista pelo modelo, de 0 a 42.

```
image_index,predicted_class
0,14
1,1
2,38
3,35
4,11
```

Caso o nome do experimento seja informado na função `save_predictions`, o arquivo poderá conter uma linha de comentário no início:

```
# experiment: CNN BatchNorm Adam Augmentation
image_index,predicted_class
0,14
1,1
2,38
3,35
4,11
```

Essa linha de comentário é permitida.

9.2 Exemplo de uso

Após obter as predições do modelo no conjunto de teste, o grupo deverá salvar o arquivo usando a função `save_predictions`.

Exemplo:

```
from gtsrb_utils import get_dataloaders, save_predictions

train_loader, val_loader, test_loader = get_dataloaders(
    img_size=32,
    batch_size=128
)

# y_pred deve conter as classes previstas pelo modelo no conjunto
# de teste
# y_pred deve ter shape (12630,)

save_predictions(
    y_pred,
    "ANA_PEDRO_RESULTS.csv",
    experiment_name="CNN BatchNorm Adam Augmentation"
)
```

O vetor `y_pred` deve conter apenas as classes previstas pelo modelo, representadas por números inteiros de 0 a 42.

10 Relatório

O relatório deve conter as seguintes seções.

10.1 Introdução

- Descrição do problema;
- Breve descrição do dataset GTSRB;
- Objetivo do trabalho.

10.2 Metodologia

- Pré-processamento;
- Divisão treino/validação/teste;
- Arquiteturas utilizadas;
- Otimizadores;
- Hiperparâmetros;
- Estratégias de augmentation;
- Métricas usadas.

10.3 Resultados

- Tabelas de resultados;
- Curvas de treino;
- Acurácia por classe;
- Matriz de confusão;
- Exemplos de imagens classificadas corretamente;
- Exemplos de imagens classificadas incorretamente.

10.4 Discussão

- Quais classes foram mais difíceis?
- O que melhorou a acurácia global?
- O que melhorou a macro accuracy?
- O que aconteceu com a pior classe?
- Houve trade-off entre melhorar classes difíceis e manter acurácia global?
- Quais erros do modelo parecem mais compreensíveis?
- Quais erros foram inesperados?

10.5 Conclusão

- Melhor modelo encontrado;
- Principais aprendizados;
- Limitações;
- Possíveis trabalhos futuros.

11 Entregáveis

Cada grupo deverá entregar:

1. Código-fonte do projeto ou notebook com os experimentos;
2. Relatório em PDF;
3. Arquivo `.csv` com as predições do melhor modelo;

12 Regras

- O arquivo auxiliar fornecido pelo professor não deve ser modificado.
- Todos os grupos devem usar a função `get_data loaders` fornecida no arquivo auxiliar para carregar treino, validação e teste.
- O conjunto de teste deve ser usado apenas para avaliação final.
- O `DataLoader` do conjunto de teste deve usar `shuffle=False`, conforme definido no arquivo auxiliar.
- O arquivo final de resultados deve ser gerado usando a função `save_predictions`.
- O arquivo final deve se chamar `NOME1_NOME2_RESULTS.csv`.
- O arquivo final deve conter uma linha para cada imagem do conjunto de teste.
- As classes previstas devem ser representadas por números inteiros de 0 a 42.
- O arquivo de resultados deve conter obrigatoriamente as colunas `image_index` e `predicted_class`.
- O grupo deve garantir que os resultados apresentados no relatório correspondam ao mesmo modelo usado para gerar o arquivo `NOME1_NOME2_RESULTS.csv`.